jigsaw-agile-community-rules

4位 writeup - Instruct LLM is all you need

Rank	4
Team	Dream Team
Author	Ruslan Berdichevsky
Topic ID	613473
Version	Japanese Translation

Source: https://www.kaggle.com/competitions/jigsaw-agile-community-rules/discussion/613473
Retrieved with Kaggle CLI on 2026-10-21.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

Kaggle Jigsaw（4位）— 技術 write-up – Instruct LLM is all you need

謝辞

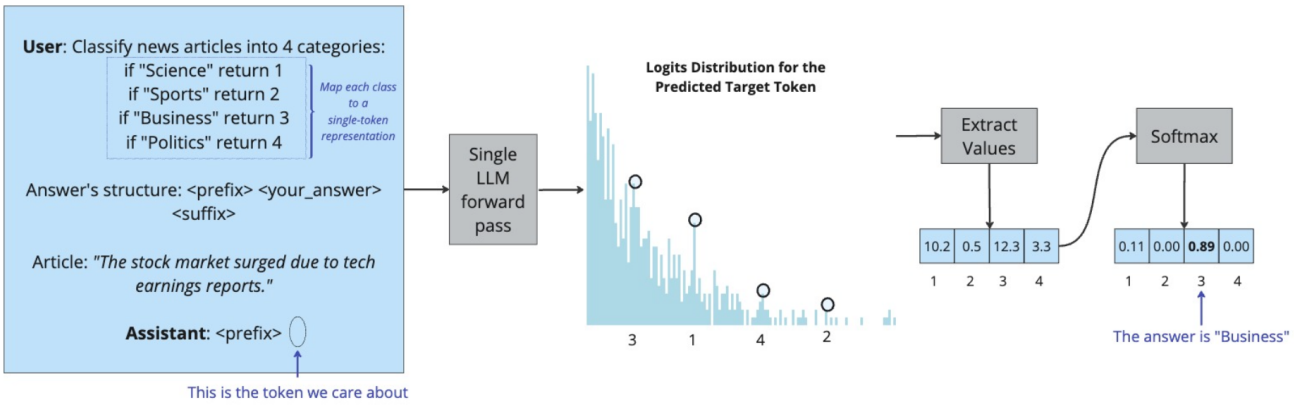
よく運営されたコンペを開催してくださった主催者の皆さんに感謝します。このコミュニティの新参者として、素晴らしい学びの経験になりました。このチャレンジはまた、instruction-tuned LLM が、調整可能なデータと組み合わさることで、強い分類性能を発揮できることを示しました。特に、**SALSA**（Single-pass Autoregressive LLM Structured Classification; 論文）[1] が力を発揮できる設定ではそうです。**SALSA は私の最近の研究で、この1年間取り組んできた研究論文です。**

概要

私の提出は **SALSA**（Single-pass Autoregressive LLM Structured Classification）に基づいています。タスクに合わせたプロンプトテンプレートと調整があれば、instruction-tuned LLM は SOTA に近い性能を達成できます。SALSA では、小さな固定の回答語彙（例: "0"、"1"）に対するモデルの次トークンの **logits** を、ラベルの **ベイズ推定量** として扱います。一般に、instruction モデルが大きく、よくアラインされているほど、SALSA の信号は強くなります。より強い zero-shot のベースラインから学習を始めるためです。AUC をもう少し絞り出すため、**Generalized Cross-Entropy (Tsallis)** と **疑似ラベル付け** の段階を導入しました。

SALSA を一段落で

- **Single pass:** 範囲を厳密に絞った質問をし、回答トークンの確率を次トークン分布から直接読み取る（複数ステップのデコードはしない）。
- **構造化されたラベル空間:** 回答を小さな語彙に制約し、logits がクラス確率にきれいに対応するようにする。
- **ベイズ的な見方:** 正規化した logits は尤度として機能する。確率として使ったり、プロンプトやルールをまたいで組み合わせたり、モデル間でブレンドしたりできる。これにより、**ソフトラベル** と **疑似ラベル付け** も効果的に使える。



データ処理

- **few-shot をラベルとして使う:** すべてのレコードには **4つ** の few-shot の例（**2つの positive** と **2つの negative**）が含まれていた。これらを、公開で提供されたラベルと並べてラベルとして使った。
- **subreddit フィールドの削除: subreddit** フィールドは誤解を招くものだったため削除した。public データでは、few-shot の例が subreddit フィールドと無関係だった。
- **重複と不一致:** 学習セットには冗長なテキストと、ときどきの不一致が含まれていた。**重複をまとめて** ユニークなデータセットを作り、ユニークなテキストごとに **ルール違反率** を計算した。学習中は、この比率を **ソフトターゲット**（同じコメントとルールの組についての positive/総数のラベルから導いた正解確率）として使った。

ベースの SALSA 学習

計算資源の制約。 Kaggle の時間予算内で約 32B パラメータ（またはそれ以上）のモデルを学習するのは大きな技術的課題だったので、それは追求しなかった。私の経験では、AWQ は GPTQ や単純な重みのクリッピングより、ビットあたりのモデル能力をよく保つため、**AWQ** のベースモデルを使った。 **試したバックボーン。** Qwen-3 **4B** と Qwen-3 **14B**。

頑健な loss。 ラベルノイズを緩和するため、cross-entropy を **Generalized Cross-Entropy (Tsallis)** に置き換え、ローカル CV と public LB のサンプリングに基づいて **q = 0.955** とした。これにより public スコアは改善したが、private スコアはわずかしか変わらなかった。効果は控えめであることを示唆している。

シードへの感度。 dataloader のシードを変えると、約 **0.002 AUC** の変動が見られた。この閾値を下回る改善（GCE によるものを含む）は、これらの制約の下では統計的に有意だと主張しにくい。

1段階での結果

Model	Private	Public
Qwen-3 4B + **GCE**	0.91874	0.92482
Qwen-3 14B + **GCE**	0.92313	0.92909
Qwen-3 14B + **CE**	0.92309	0.92803

プロンプトと推論

大きなモデルを予算内に収めるため、**短いテンプレート** を使い、**prefix KV-cache** の再利用を最大化するため **コメントを最後に** 置いた。推論時は、ルールのテキストもキャッシュに残るよう **ルールごとにバッチ化** した。

プロンプトテンプレート

```
<|im_start|>user

Does the comment violate the rule? Answer  the :
<ANSWER>
Where the number  one of:
 - No
 - Yes
```

```
The rule:
<RULE>
{rule}
</RULE>

The Comment:
<COMMENT>
{body}
</COMMENT>
<|im_end|>
<|im_start|>assistant
<think>

</think>
<ANSWER>
```

小さい方の Qwen-3 4B では、指示を **最後** に移すと非常に小さな改善があったが、この設定では統計的有意性を下回るものだった。

疑似ラベルを使った2段階学習

同じ計算資源の範囲内で AUC をさらに押し上げるため:

1. **Stage 1（小さいモデル）。** 小さいモデルを学習し、対象のテストドメインで推論を実行する。
2. **不確実性サンプリング。** 最も不確実な（最も情報量の多い）テストサンプル **約8,000件** を、Stage 2 用のソフトな **疑似ラベル** として選ぶ。
   - これらの疑似ラベルは **新しい外部知識を追加するものではない**。むしろ、テストサンプル間の **相対的なスコアを固定する（anchor する）** ことで、instruction モデルの事前学習済みの知識をより効果的に使えるようにする。
3. **Stage 2（大きいモデル）。** 元のデータ **に加えて** 疑似ラベル付きのセットで **Qwen-3 14B** をファインチューニングする。
4. **ブレンド。** Stage-1 と Stage-2 の予測を、重み **0.8 : 0.2** でブレンドする。

最終結果

System / Ablation	Private	Public
優勝ソリューション（ブレンド 0.8 / 0.2）	**0.9286**	**0.93382**
ブレンドなし（Stage-2 のみ）	0.9264	0.93111
Qwen-3 4B（1段階）	0.91874	0.92482
Qwen-3 14B（1段階）	0.92313	0.92909
Qwen-3 4B での2段階	0.92239	0.92642
Qwen-3 4B（1段階）と Qwen-3 14B（1段階）のブレンド	0.92469	0.92921

実務上の注意。 2段階のパイプラインはエンジニアリングの複雑さを増やした。ここでは約 **0.004 AUC** の改善が順位を決定づけたが、それが「割に合う」かどうかは、チームの時間・計算資源の予算次第である。

参考文献

[1] **SALSA: Single-pass Autoregressive LLM Structured Classification.**